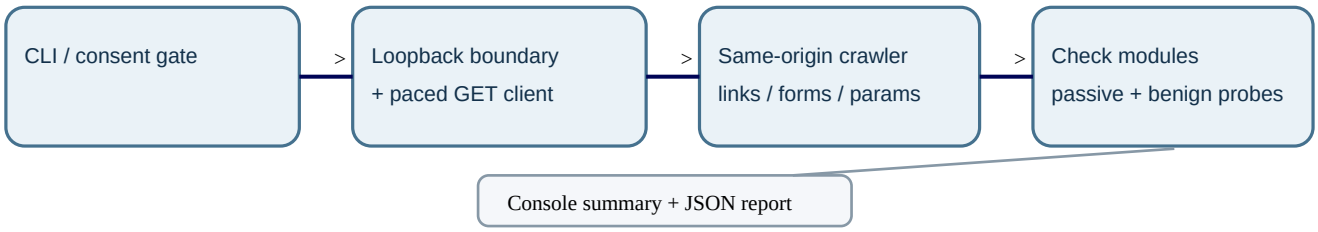
Lightweight Web Vulnerability Scanner

Design, safety controls, detection coverage, and validation status

Purpose and scope

This project implements a small Python command-line scanner for authorised localhost web labs. It crawls same-origin pages, records links, forms and query parameters, runs passive configuration checks and narrowly scoped harmless GET probes, and writes console and JSON results. The assignment requires a consent gate, safe request pacing, a page limit, timeouts, a clear User-Agent, 10 or more checks across at least six OWASP categories, unit tests, and scan evidence for two lab targets.

Architecture



Safety controls

The scanner refuses execution without **--i-have-permission** and accepts only literal loopback IP addresses. All requests use GET with redirects disabled, a 250 ms default delay, an 8-second timeout and a 30-page cap. Form submissions are never sent. The reflected marker is unique text, and SQL probing is a single quote matched against database-specific error signatures. File probes require content signatures, not status code alone.

Current execution status

Both sample targets were the workspace's local vulnerable SecureShop labs. Flask ran at 127.0.0.1:5000; Node/Express ran at 127.0.0.1:18080 because its default port was occupied. Both scans completed without connection errors. These are indicators from an unauthenticated crawl, not a complete assessment.

Check coverage and OWASP mapping

The implementation includes the following detection modules. A finding is an indicator that needs human confirmation; the scanner does not assert exploitability.

Check	OWASP	Evidence / confidence limit
HTTP instead of HTTPS; missing HSTS	A02	Scheme and header inspection; HSTS is relevant only over HTTPS.
Cookie Secure flag	A02	Cookie attribute check on HTTPS responses.
Missing CSP, X-Frame-Options, nosniff	A05	Response header absence.
Directory listing	A05	Known directory-index title signature.
Exposed .env, .git/HEAD, phpinfo	A05	GET path probes require characteristic content.
SQL error after quote probe	A03	Database error text; may miss blind or suppressed errors.
Reflected harmless marker	A03	Unique marker reflected in HTML; reflection alone does not prove execution.
Server version disclosure	A06	Banner is an inventory signal, not a CVE match.
HttpOnly and SameSite cookie flags	A07	Set-Cookie attribute inspection.
External script without SRI	A08	External script tag lacks integrity attribute.
Potential tokenless state-changing form	A01	Passive field-name heuristic; does not submit form.
Permissive CORS; mixed content	A01 / A02	Header policy and insecure resource reference inspection.

The scanner covers A01, A02, A03, A05, A06, A07 and A08. A09 (Security Logging and Monitoring Failures) and A10 (Server-Side Request Forgery) are not tested remotely. Outdated dependencies require a package inventory and advisory database; a version banner alone is insufficient.

Sample scan outcomes and accuracy

Both JSON reports were generated from completed scans of vulnerable localhost lab applications. Findings cover only pages reachable without authentication and require manual review.

Lab target	Pages	Findings	Observed indicators
Flask SecureShop 127.0.0.1:5000	8	5: 4 medium, 1 low	HTTP; missing CSP, X-Frame-Options and nosniff; version banner.
Node SecureShop 127.0.0.1:18080	13	5: 4 medium, 1 low	HTTP; missing CSP, X-Frame-Options and nosniff; reflected marker.

Both reports have status **complete**. Missing headers and version disclosure are configuration evidence; the reflected-marker alert is low confidence and does not demonstrate script execution. Neither scan found an exposed-file signature or database error.

Known limitations observed in this run

An earlier Flask manual validation record reports an HttpOnly-missing session cookie, SQL-injection behavior on product search, and cross-account order/profile exposure. This scanner did not detect those: its crawl was unauthenticated, no session cookie was set on the visited public pages, and its quote request did not return a recognized database error. The Node lab documentation describes an old lodash pin, but this scanner does not inventory dependencies or query CVE databases. These are known blind spots, not contradictions.

False positives and false negatives

False positives: A missing token-like field name does not prove CSRF vulnerability because tokens may be supplied through headers or framework middleware. Version banners can be stale or backported. Generic SQL error wording can arise from application handling. CORS wildcard is not automatically exploitable for credentialed requests. An external script without SRI may be intentionally controlled by the application. Findings are deduplicated by check ID and URL and include bounded evidence for review.

False negatives: Page limits and link-only crawling miss unlinked and authenticated routes. HTML parsing does not run JavaScript. One marker and one quote miss encoded, stored, DOM-based, blind and context-specific injection. GET probes cannot discover state-dependent access-control defects, SSRF or business-logic flaws. Redirects are not followed, some content types are skipped, and response limits can truncate evidence. These limits should be reported with every scan.

Remediation and demo plan

Recommended fixes

• Serve all pages over HTTPS; add HSTS after validating HTTPS deployment and set Secure, HttpOnly and appropriate SameSite cookie attributes.

• Set a restrictive Content-Security-Policy, frame-ancestors or X-Frame-Options, and X-Content-Type-Options: nosniff. Disable directory indexing and remove/block development files such as .env, .git and phpinfo.

• Use parameterized SQL and context-aware output encoding; do not treat a scanner marker response as proof of script execution.

• Allowlist CORS origins, add CSRF middleware to state-changing actions, pin and update dependencies, and remove unnecessary server version banners.

• Review every alert against application behavior and compare with an OWASP ZAP baseline scan on the same lab profile. Record tool versions, scan date, mode, paths and confirmed/false-positive status.

Five-minute demonstration

1. Start each intentionally vulnerable app on its documented loopback address. 2. Show that invocation without the permission flag is rejected and a remote hostname is refused. 3. Scan the Flask lab with a small page cap and explain request delay, evidence and JSON output. 4. Run the Node lab and compare findings. 5. Discuss one confirmed header or cookie issue, one potential false positive, and what the tool cannot test. Use only labs you are authorised to assess.

Local rerun commands

```
python scanner.py http://127.0.0.1:5000 --i-have-permission -o reports/secureshop-flask.json
python scanner.py http://127.0.0.1:18080 --i-have-permission -o reports/secureshop-node.json
```

Project files: scanner.py, README.md, requirements.txt, tests/test_scanner.py, and reports/. Unit tests cover header checks, cookie flags, SQL error signatures, and loopback enforcement.